

# Entrega 1 - Proyecto Intermodular DAW

---

## Plataforma de Monitoreo de Sensores Industriales (IIAVA)

**Alumno:** Juan Diego Martin-Blas Ramos

**Ciclo:** Desarrollo de Aplicaciones Web (DAW) - Semipresencial

**Centro:** IES L'Estacio - Ontinyent

**Curso:** 2025-26

**Fecha de entrega:** Semana del 16 de marzo de 2026

## Indice

---

1. Introduccion del proyecto
2. Objetivos de la Entrega 1
3. Arquitectura del sistema
4. Funcionalidades implementadas
  - 4.1 Gestion de sensores
  - 4.2 Recopilacion y visualizacion de datos
  - 4.3 Sistema de alertas
  - 4.4 Notificaciones en tiempo real (SSE)
  - 4.5 Autenticacion y autorizacion
  - 4.6 Envio de datos y geolocalizacion
5. Stack tecnologico
6. Modelo de datos
7. API REST - Endpoints
8. Despliegue en produccion
9. Testing
10. Estado actual y proximos pasos

# 1. Introduccion del proyecto

## Descripcion general

La **Plataforma de Monitoreo de Sensores Industriales (IIAVA)** es un sistema web full-stack diseñado para proporcionar visibilidad centralizada y en tiempo real de maquinas industriales y equipos de fabrica a traves de la recopilacion y visualizacion de datos de sensores.

## El problema

Las instalaciones industriales modernas cuentan con numerosas maquinas y equipos dotados de multiples sensores que miden parametros criticos como temperatura, presion, vibracion y otras metricas. Sin embargo, la gestion y el monitoreo de estos sensores suele estar fragmentado:

- **Informacion dispersa:** Los datos de sensores estan aislados en maquinas o sistemas individuales.
- **Falta de visibilidad:** No existe una vista centralizada de todos los sensores de la instalacion.
- **Monitoreo manual:** Los operadores deben verificar fisicamente el equipo o navegar por multiples sistemas.
- **Respuesta retardada:** Los problemas pueden pasar desapercibidos hasta que ocurre una falla del equipo.
- **Silos de datos:** Diferentes maquinas utilizan diferentes sistemas de monitoreo.

## Nuestra solucion

Una plataforma integrada basada en web que proporciona:

- **Gestion centralizada de sensores:** Una interfaz unica para registrar, ver y gestionar todos los sensores de la instalacion.
- **Panel de monitoreo en tiempo real:** Interfaz responsive que muestra el estado de todos los sensores con indicadores visuales.
- **Sistema de alertas inteligente:** Configuracion de umbrales y condiciones que disparan alertas automaticas.
- **Notificaciones en tiempo real:** Actualizaciones instantaneas mediante Server-Sent Events (SSE).
- **Visualizacion de datos historicos:** Graficos interactivos con filtrado por rango de fechas.

## Audiencia objetivo

| Perfil                   | Uso principal                                   |
|--------------------------|-------------------------------------------------|
| Gerentes de fabrica      | Monitorear la salud general de la instalacion   |
| Equipos de mantenimiento | Rastrear sensores para mantenimiento predictivo |
| Personal de operaciones  | Ver metricas de produccion en tiempo real       |
| Aseguramiento de calidad | Monitorear parametros de calidad                |
| Ingenieros industriales  | Analizar datos de sensores para optimizacion    |

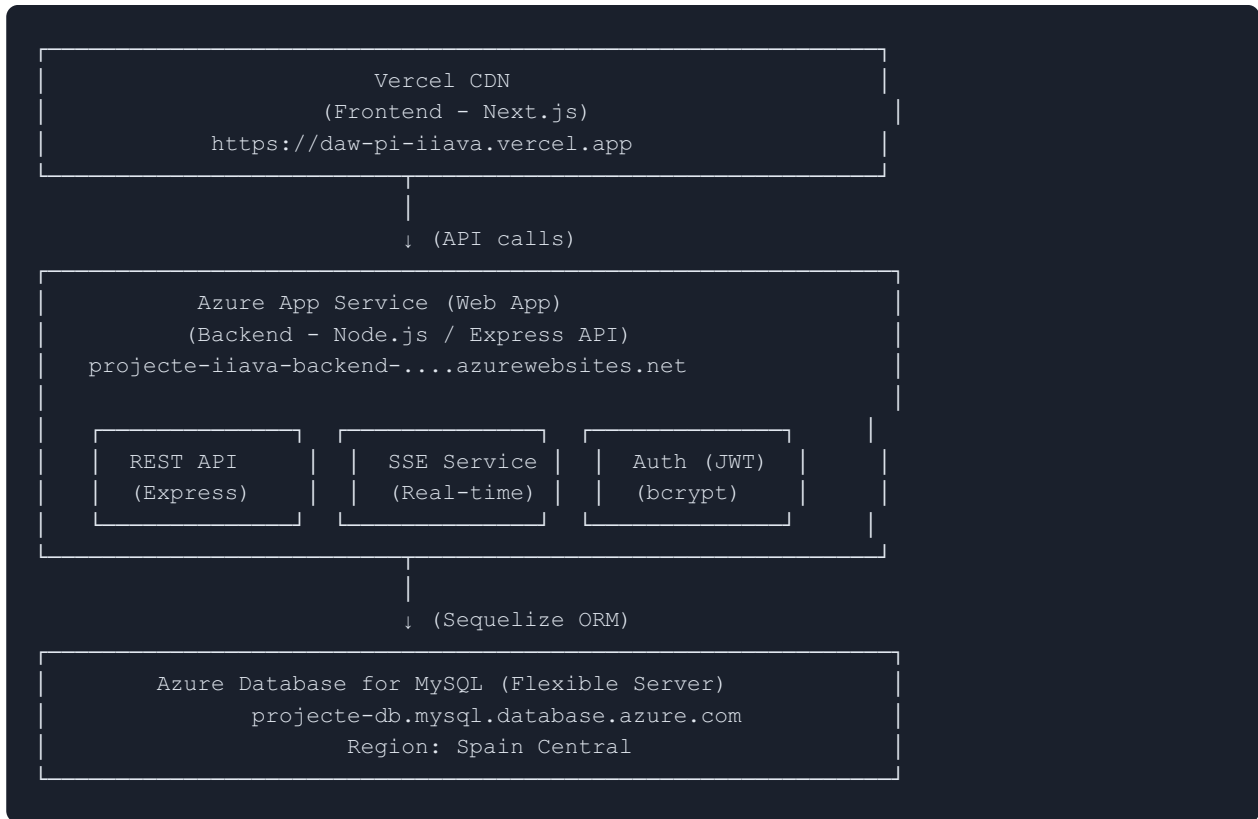
## 2. Objetivos de la Entrega 1

La **Entrega 1** tiene como objetivo demostrar que la base del proyecto esta completa y funcional, con las siguientes funcionalidades operativas:

| Funcionalidad                 | Descripcion                                             | Estado     |
|-------------------------------|---------------------------------------------------------|------------|
| Gestion de sensores (CRUD)    | Crear, leer, actualizar y eliminar sensores             | Completado |
| Recopilacion de datos         | Ingesta de datapoints con validacion de tipo            | Completado |
| Visualizacion de datos        | Graficos interactivos con filtrado temporal             | Completado |
| Sistema de alertas            | Alertas configurables con condiciones logicas           | Completado |
| Notificaciones en tiempo real | Server-Sent Events para actualizaciones instantaneas    | Completado |
| Autenticacion                 | Registro, login, JWT, roles de usuario                  | Completado |
| Despliegue en la nube         | Frontend en Vercel, backend en Azure, BD en Azure MySQL | Completado |
| CI/CD                         | Despliegue automatizado con GitHub Actions              | Completado |
| Testing                       | Tests de integracion en frontend y backend              | Completado |

### 3. Arquitectura del sistema

#### Diagrama de arquitectura en produccion



#### Patrones de comunicacion

- **Cliente → Servidor**: Peticiones HTTP REST (JSON)
- **Servidor → Cliente**: Server-Sent Events (SSE) para actualizaciones en tiempo real
- **Autenticacion**: JWT almacenado en cookies `httpOnly` con expiracion de 7 dias
- **ORM**: Sequelize gestiona el mapeo objeto-relacional contra MySQL

## 4. Funcionalidades implementadas

---

### 4.1 Gestion de sensores

**CRUD completo** para la gestion de sensores industriales.

Cada sensor se define con:

- **Alias:** Nombre descriptivo del sensor (ej: "Temperatura Motor Cinta Transportadora")
- **Tipo de dato:** `int`, `float`, `boolean`, `string`
- **Metadatos automaticos:** Fechas de creacion y ultima actualizacion

**Interfaz de usuario:**

- Lista de sensores con indicadores visuales de tipo (etiquetas con codigo de color)
- Formulario de creacion con seleccion de tipo
- Formulario de edicion (alias y tipo)
- Eliminacion con dialogo de confirmacion
- Navegacion a detalle de datapoints desde cada sensor

### 4.2 Recopilacion y visualizacion de datos

#### Datapoints

Los sensores reciben datos a traves de la API REST. Cada datapoint se almacena con validacion de tipo estricta:

- Los valores se almacenan en columnas especificas segun el tipo ( `valueInt`, `valueFloat`, `valueBoolean`, `valueString` )
- Indexacion por `sensorId` y `timestamp` para consultas eficientes
- Filtrado por rango de fechas mediante parametros `from` y `to`
- Los parametros de filtro se persisten en la URL para compartir vistas

#### Visualizacion con graficos interactivos

Implementados con la libreria **Recharts**:

- **LineChart:** Series temporales para sensores numericos (int/float)
- **AreaChart:** Con gradientes para visualizacion de umbrales de alerta
- **BarChart:** Comparativas de datos entre periodos
- Deteccion inteligente de decimales para valores float
- Timestamps con precision adaptativa (incluye segundos cuando es necesario)
- Visualizacion de zonas de alerta con colores de gradiente

#### Envio manual de datos

Pagina dedicada ( `/send-data` ) con:

- Formulario para envio manual de datapoints
- Selector de sensor
- Campo de valor adaptado al tipo del sensor (numerico, booleano, texto)
- Feedback de exito/error tras el envio

### 4.3 Sistema de alertas

CRUD completo de alertas vinculadas a sensores con evaluacion automatica:

- **Condiciones soportadas:** `>`, `<`, `>=`, `<=`, `==`, `!=`
- **Valor umbral:** Valor numerico de referencia para la condicion
- **Descripcion:** Texto descriptivo opcional
- **Habilitacion:** Activar/desactivar alertas individualmente
- **Evaluacion automatica:** Cada vez que se crea un datapoint, se evaluan todas las alertas activas del sensor. Si se cumple la condicion, se dispara un evento SSE de tipo `alert-triggered`

**Interfaz de usuario ( `/alerts` ):**

- Lista de alertas con estado visual (activa/inactiva)
- Formulario de creacion vinculado a un sensor
- Edicion y eliminacion de alertas

### 4.4 Notificaciones en tiempo real (SSE)

El sistema de notificaciones proporciona actualizaciones instantaneas sin polling mediante **Server-Sent Events**:

**Backend - SSE Service ( `sseService.js` ):**

- Endpoint SSE en `/api/sensors/events`
- Broadcast de eventos a todos los clientes conectados
- Tipos de eventos emitidos:
  - `datapoint-created` : Cuando se registra un nuevo dato en un sensor
  - `alert-triggered` : Cuando un datapoint cumple una condicion de alerta
- Gestion de conexiones con reconexion automatica

**Frontend - NotificationListener:**

- Componente React que suscribe al endpoint SSE a traves de un proxy en Next.js ( `/api/sensors/events` )
- Notificaciones visuales en tiempo real
- Integracion con el sistema de alertas para mostrar avisos criticos

### 4.5 Autenticacion y autorizacion

Sistema de autenticacion completo:

- **Registro:** Creacion de cuentas con validacion de datos
- **Hash de contraseñas:** bcrypt para almacenamiento seguro
- **Login:** Generacion de JWT con expiracion de 7 dias
- **Sesiones:** JWT almacenado en cookies httpOnly (proteccion contra XSS)
- **Roles:** `user` y `admin` (preparado para control de acceso granular)
- **Rutas protegidas:** Componente `ProtectedRoute` en frontend; verificacion de token en backend
- **Cierre de sesion:** Limpieza de cookie JWT

**Interfaz de usuario ( `/login` ):**

- Formulario dual de registro/login
- Redireccion automatica tras autenticacion
- Header con indicador de usuario autenticado

## 4.6 Envio de datos y geolocalizacion

**Pagina de envio de datos** ( `/send-data` ):

- Formulario web para envio manual de lecturas de sensores
- Validacion de tipo antes del envio
- Selector de sensor con tipo visible
- Feedback visual de exito o error

**Geolocalizacion** ( `/geolocation` ) - Funcionalidad experimental:

- Seguimiento de ubicacion en tiempo real del dispositivo
- Seleccion de propiedad a rastrear (latitud, longitud, altitud, velocidad, precision, rumbo)
- Modo de seguimiento continuo con `watchPosition`
- Integracion con datapoints de sensores para registro de ubicacion



## 5. Stack tecnologico

### Frontend

| Tecnologia      | Version | Proposito                                            |
|-----------------|---------|------------------------------------------------------|
| React           | 19.2.0  | Biblioteca de UI con componentes funcionales y hooks |
| Next.js         | 15.3.4  | Framework con SSR, routing y API routes              |
| Tailwind CSS    | 4.1     | Estilos responsive utility-first                     |
| Recharts        | 3.3.0   | Graficos interactivos para visualizacion de datos    |
| Vitest          | 4.0.8   | Framework de testing rapido                          |
| Testing Library | -       | Testing de componentes desde perspectiva del usuario |

### Backend

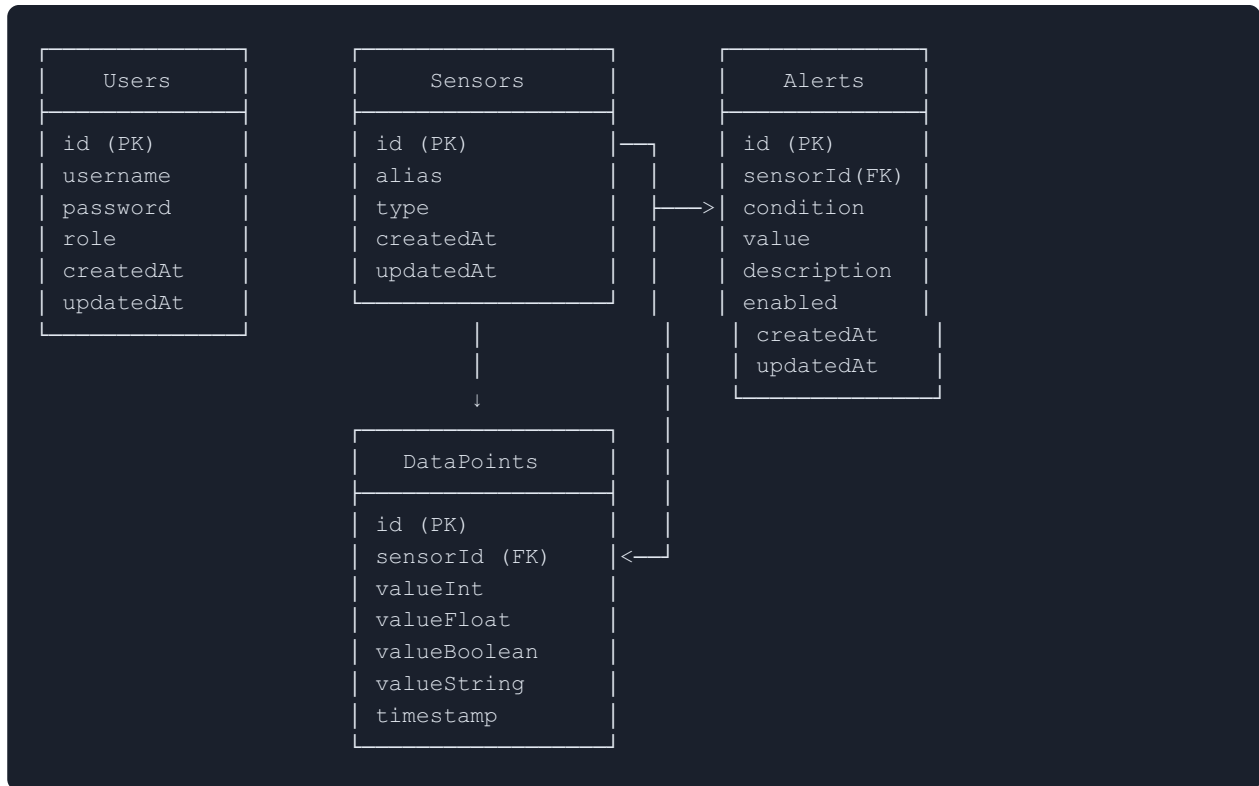
| Tecnologia         | Version | Proposito                                              |
|--------------------|---------|--------------------------------------------------------|
| Node.js            | 22      | Runtime de servidor                                    |
| Express.js         | 5.1.0   | Framework REST API                                     |
| Sequelize          | 6.37.0  | ORM para MySQL con migraciones automaticas             |
| MySQL              | 8.0     | Base de datos relacional en produccion                 |
| SQLite             | -       | Base de datos en memoria para tests                    |
| JWT (jsonwebtoken) | -       | Generacion y verificacion de tokens de autentificacion |
| bcryptjs           | -       | Hash seguro de contraseñas                             |
| Pino               | 10.1.0  | Logging estructurado de alto rendimiento               |

## DevOps e Infraestructura

| Tecnologia               | Proposito                                               |
|--------------------------|---------------------------------------------------------|
| Docker & Docker Compose  | Contenedorizacion para desarrollo y produccion          |
| GitHub Actions           | CI/CD: build, test y deploy automatizados               |
| Vercel                   | Hosting frontend con CDN global y deploy automatico     |
| Azure App Service        | Hosting backend (Node.js en Linux)                      |
| Azure Database for MySQL | Base de datos gestionada (Flexible Server)              |
| Makefile                 | +20 comandos para automatizar operaciones de desarrollo |

## 6. Modelo de datos

### Diagrama Entidad-Relacion



### Descripción de tablas

#### Sensors

- **id**: Clave primaria autoincremental
- **alias**: Nombre descriptivo del sensor (obligatorio)
- **type**: Tipo de dato - enum: `int`, `float`, `boolean`, `string`
- **createdAt**, **updatedAt**: Timestamps automáticos de Sequelize

#### DataPoints

- **id**: Clave primaria autoincremental
- **sensorId**: Clave foránea → **Sensors** (CASCADE en eliminación)
- **valueInt**, **valueFloat**, **valueBoolean**, **valueString**: Columnas tipadas (solo se usa la correspondiente al tipo del sensor)
- **timestamp**: Momento de la lectura
- **Indices**: `sensorId`, `timestamp`, compuesto `sensorId + timestamp` (optimización de consultas por rango)

#### Alerts

- **id**: Clave primaria autoincremental
- **sensorId**: Clave foránea → **Sensors** (CASCADE en eliminación)
- **condition**: Operador de comparación - enum: `>`, `<`, `>=`, `<=`, `==`, `!=`
- **value**: Valor umbral numérico (float)
- **description**: Descripción textual (opcional)

- `enabled` : Booleano para activar/desactivar (defecto: `true` )

## Users

- `id` : Clave primaria autoincremental
- `username` : Nombre de usuario (unico, obligatorio)
- `password` : Contraseña hasheada con bcrypt (nunca en texto plano)
- `role` : Rol del usuario - enum: `user` , `admin` (defecto: `user` )

## Relaciones

| Relacion            | Tipo         | Comportamiento al eliminar           |
|---------------------|--------------|--------------------------------------|
| Sensor → DataPoints | Uno a muchos | CASCADE (se eliminan los datapoints) |
| Sensor → Alerts     | Uno a muchos | CASCADE (se eliminan las alertas)    |

## 7. API REST - Endpoints

### Autenticacion ( /api/auth )

| Metodo | Endpoint           | Descripcion                             | Autenticacion |
|--------|--------------------|-----------------------------------------|---------------|
| POST   | /api/auth/register | Registro de nuevo usuario               | No            |
| POST   | /api/auth/login    | Login (devuelve JWT en cookie httpOnly) | No            |
| GET    | /api/auth/me       | Obtener datos del usuario actual        | Si (JWT)      |
| POST   | /api/auth/logout   | Cerrar sesion (limpia cookie)           | Si (JWT)      |

### Sensores ( /api/sensors )

| Metodo | Endpoint         | Descripcion                       | Autenticacion |
|--------|------------------|-----------------------------------|---------------|
| GET    | /api/sensors     | Listar todos los sensores         | Si            |
| POST   | /api/sensors     | Crear nuevo sensor (alias, type)  | Si            |
| PUT    | /api/sensors/:id | Actualizar sensor                 | Si            |
| DELETE | /api/sensors/:id | Eliminar sensor y datos asociados | Si            |

### Datapoints ( /api/sensors/:id/datapoints )

| Metodo | Endpoint                    | Descripcion                                                                      | Autenticacion |
|--------|-----------------------------|----------------------------------------------------------------------------------|---------------|
| GET    | /api/sensors/:id/datapoints | Obtener datapoints. Query params: <code>from</code> , <code>to</code> (ISO 8601) | Si            |
| POST   | /api/sensors/:id/datapoints | Crear datapoint. Valida tipo contra el sensor. Dispara evaluacion de alertas     | Si            |

**Alertas ( /api/alerts )**

| Metodo | Endpoint        | Descripcion                                                 | Autenticacion |
|--------|-----------------|-------------------------------------------------------------|---------------|
| GET    | /api/alerts     | Listar alertas. Query param opcional: <code>sensorId</code> | Si            |
| POST   | /api/alerts     | Crear alerta (sensorId, condition, value, description)      | Si            |
| PUT    | /api/alerts/:id | Actualizar alerta                                           | Si            |
| DELETE | /api/alerts/:id | Eliminar alerta                                             | Si            |

**Eventos en tiempo real**

| Metodo | Endpoint            | Descripcion                      | Protocolo |
|--------|---------------------|----------------------------------|-----------|
| GET    | /api/sensors/events | Stream de eventos en tiempo real | SSE       |

**Eventos emitidos:**

- `datapoint-created`: { `sensorId`, `value`, `timestamp` }
- `alert-triggered`: { `alertId`, `sensorId`, `condition`, `value`, `datapoint` }

## 8. Despliegue en produccion

### Infraestructura

La aplicacion esta desplegada en produccion y es accesible desde Internet:

| Componente    | Plataforma                  | URL                                                                               | Estado |
|---------------|-----------------------------|-----------------------------------------------------------------------------------|--------|
| Frontend      | Vercel (CDN global)         | https://daw-pi-iiava.vercel.app                                                   | Activo |
| Backend       | Azure App Service           | https://projecte-iiava-backend-eue7f0eghzakbkcd.spaincentral-01.azurewebsites.net | Activo |
| Base de Datos | Azure MySQL Flexible Server | projecte-db.mysql.database.azure.com (Spain Central)                              | Activo |

### CI/CD con GitHub Actions

El despliegue se realiza de forma completamente automatizada:

1. **Push a `main`** → Se disparan los workflows de GitHub Actions
2. **Frontend:** Vercel detecta cambios en `/frontend` y despliega automaticamente a su CDN global
3. **Backend:** GitHub Actions ejecuta build y despliega en Azure App Service
4. **Base de datos:** Sequelize sincroniza el esquema automaticamente al arrancar el servidor ( `sync()` )

### Contenerizacion con Docker

El proyecto esta completamente contenedorizado:

- `Dockerfile` y `Dockerfile.dev` para frontend y backend (entornos separados)
- `docker-compose.yml` para produccion
- `docker-compose.dev.yml` para desarrollo (con hot-reload via volumenenes)
- `Makefile` con +20 comandos para automatizar operaciones ( `make dev-up` , `make prod-build` , etc.)

## 9. Testing

### Estrategia

Se prioriza el **testing de integracion** que prueba el comportamiento de la aplicacion desde la perspectiva del usuario final, en lugar de tests unitarios aislados.

### Tests del Backend (Vitest + Supertest)

| Archivo de test                              | Funcionalidad cubierta                                      |
|----------------------------------------------|-------------------------------------------------------------|
| <code>sensors.datapoints.post.test.js</code> | Creacion de datapoints con validacion estricta de tipo      |
| <code>sensors.datapoints.test.js</code>      | Consulta de datapoints con filtros de rango de fechas       |
| <code>sensors.update.test.js</code>          | Actualizacion de alias y tipo de sensores                   |
| <code>sensors.delete.test.js</code>          | Eliminacion de sensores con cascade de datapoints y alertas |

- **Base de datos en memoria:** SQLite para aislamiento total entre tests
- **Tests end-to-end de API:** Supertest simula peticiones HTTP reales contra Express

### Tests del Frontend (Vitest + React Testing Library)

| Archivo de test                                   | Funcionalidad cubierta                             |
|---------------------------------------------------|----------------------------------------------------|
| <code>Sensors.list.test.jsx</code>                | Listado de sensores, indicadores de tipo, acciones |
| <code>Sensors.create.test.jsx</code>              | Formulario de creacion con validacion              |
| <code>Sensor.edit.test.jsx</code>                 | Edicion de sensores existentes                     |
| <code>Sensor.delete.test.jsx</code>               | Eliminacion con dialogo de confirmacion            |
| <code>SensorDataPoints.test.jsx</code>            | Renderizado de graficos con datos reales           |
| <code>SensorDataPoints.navigation.test.jsx</code> | Navegacion entre vistas de datos                   |

- **Mocks de Next.js:** Router y modulos mockeados para testing aislado
- **Perspectiva del usuario:** Queries por rol, texto visible y comportamiento, no por implementacion interna



## 10. Estado actual y proximos pasos

### Resumen del estado de la Entrega 1

| Funcionalidad                               | Estado     | Progreso |
|---------------------------------------------|------------|----------|
| Gestion de sensores (CRUD completo)         | Completado | 100%     |
| Recopilacion de datos (Datapoints)          | Completado | 100%     |
| Visualizacion con graficos interactivos     | Completado | 100%     |
| Sistema de alertas configurables            | Completado | 100%     |
| Notificaciones en tiempo real (SSE)         | Completado | 100%     |
| Autenticacion y autorizacion (JWT)          | Completado | 100%     |
| Envio manual de datos                       | Completado | 100%     |
| Geolocalizacion (experimental)              | Completado | 100%     |
| Despliegue en produccion (Azure + Vercel)   | Completado | 100%     |
| CI/CD automatizado (GitHub Actions)         | Completado | 100%     |
| Testing de integracion (frontend + backend) | Completado | 100%     |

### Proximos pasos (Entrega 2 - Incidencias)

Para la proxima entrega se planifica:

1. **Sistema de incidencias:** CRUD completo para gestionar incidencias vinculadas a sensores y alertas
2. **Flujo de trabajo de incidencias:** Estados (abierta, en progreso, resuelta), asignacion a usuarios, historial de cambios
3. **Dashboard de incidencias:** Vista general con filtros por estado, sensor y fecha
4. **Mejoras en el sistema de alertas:** Integracion de alertas con la creacion automatica de incidencias

### Calendario restante

| Fecha           | Entrega              | Contenido                       |
|-----------------|----------------------|---------------------------------|
| Semana 13 abril | <b>Entrega 2</b>     | Incidencias                     |
| Semana 11 mayo  | <b>Entrega 3</b>     | Memoria del proyecto            |
| Semana 25 mayo  | <b>Entrega final</b> | Entrega completa + Presentacion |

## Repositorio y acceso

| Recurso                  | Enlace                                                                                                                                                                            |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Repositorio GitHub       | <a href="https://github.com/juandiegombr/daw.pi.iiava">https://github.com/juandiegombr/daw.pi.iiava</a>                                                                           |
| Aplicacion en produccion | <a href="https://daw-pi-iiava.vercel.app">https://daw-pi-iiava.vercel.app</a>                                                                                                     |
| API Backend              | <a href="https://projecte-iiava-backend-eue7f0eghzakbkcd.spaincentral-01.azurewebsites.net">https://projecte-iiava-backend-eue7f0eghzakbkcd.spaincentral-01.azurewebsites.net</a> |

*Documento de la Entrega 1 del Proyecto Intermodular - DAW 2025-26 IES L'Estacio - Ontinyent*